

ITSS SOFTWARE DEVELOPMENT/ SOFTWARE DESIGN AND CONSTRUCTION

Lab 03: Software Requirement Specification

Lecturer: NGUYEN Thi Thu Trang, trangntt@soict.hust.edu.vn

1. SUBMISSION GUIDELINES

To submit this lab, you and your teammate have to push all the work to the group GitHub repository, following the naming convention `MSTeamsName-TeamNumber` (e.g., TKXDPM.VP.20251-01).

For this lab, you are required to check out a branch named `release/lab03`, contains all tasks related to use case specifications, activity diagrams and the “Software Requirement Specification” document.

The deadline for submission will be updated by the lecturer or the teaching assistant. Therefore, you need to pay attention to avoid late submissions.

2. OVERVIEW

In the previous labs, you have already become familiar with use case diagrams, creating outlines for use cases, and writing use case specifications. In this lab, you will apply that knowledge to complete the “Software Requirement Specification” (SRS) document together with your team.

First, you are required to carefully discuss with your teammates to decide which use case each member will be responsible for (e.g., member A is responsible for the use case “Place order”, member B for “Pay order”), according to the lecturer’s requirement. Afterwards, you will write the specifications for the use cases you are in charge of, following the guidelines provided in **Section 3**.

Then, based on the specifications, you will then draw the corresponding activity diagrams for your use cases as instructed in **Section 4**.

Finally, you and your team will consolidate all the work done from Lab 1 up to now into a single document named “Software Requirement Specification”. You are expected to complete all the required sections of this document according to the guidelines in **Section 5**, marking the conclusion of the requirement analysis process in the software development process.

3. USE CASE SPECIFICATION

In the previous labs, you have practiced creating outlines and writing use case specifications based on a given template. However, those exercises were limited to sample use cases such as “Place Order” and “Pay Order”. Starting from this lab, you will work in teams, with each member responsible for specific use cases. Therefore, your task in this lab is to complete the outlines and write the specifications for the use cases that you are assigned.

You are required to create a directory named `RequirementAnalysis/UCSpec/` to submit your work. Please note that you must submit both the WORD file (in DOC or DOCX format) and the exported PDF file. Remember to name the files according to the corresponding use case.

```
RequirementAnalysis/  
└─ UCSpec/  
    └─ word/  
        └─ PlaceOrder.docx  
        └─ ... (put your files here)  
    └─ pdf/  
        └─ PlaceOrder.pdf  
        └─ ... (put your files here)
```

4. ACTIVITY DIAGRAM

First, you can see the following link to know how to make an activity diagram with Astah UML: <https://astah.net/support/astah-pro/user-guide/activity-diagram/>.

After specifying the use case, you should have a thorough understanding of its flow of events. An activity diagram can then be used to provide a clear, visual representation of this flow, making it easier to understand the sequence of actions, decision points, and parallel processes within the use case.

Here are some tips to help you create a complete activity diagram, illustrated using the use case “Place Order”.

4.1. Identify the required swimlane(s)/partition(s)

You will determine the actors or components involved in the process. In use case “Place Order”, they are *Customer* and *AIMS Software*.

In Asta UML, there are two types of swimlane/partition: horizontal and vertical. You may choose either type, depending on your intended purpose.

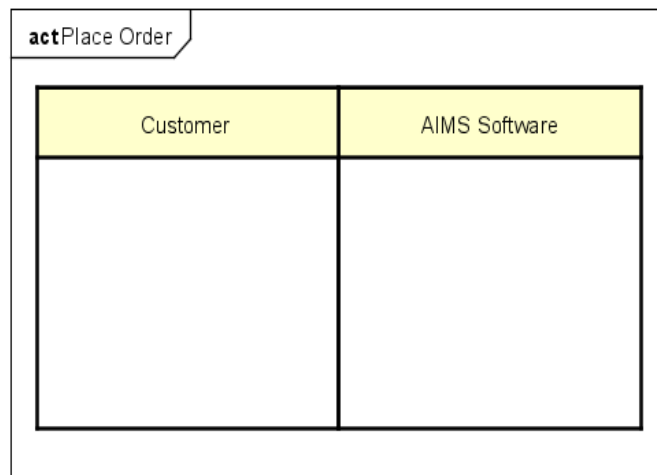


Figure 1. Vertical Partition

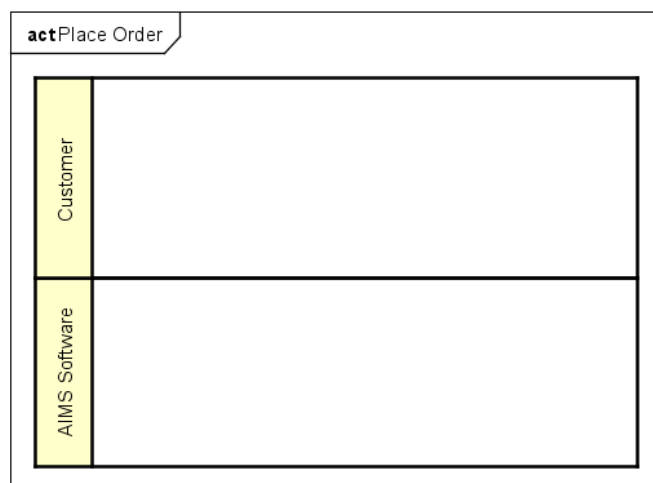


Figure 2. Horizontal Partition

4.2. Identify the start (initial) node

This node represents the beginning of the process with a solid black circle. Notice that: **An activity diagram can have only one start node.** In use case “Place Order”, the process begins when the customer requests to place an order (e.g., by pressing the “Place order” button on cart screen).

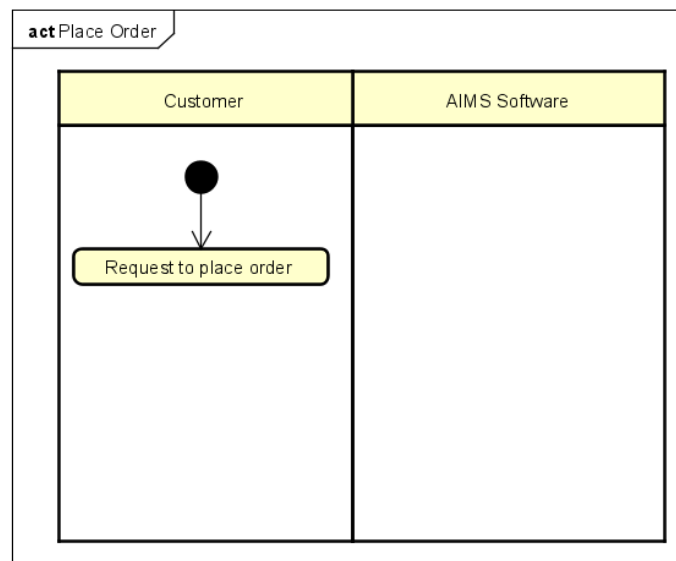


Figure 3. Initial Node of Activity Diagram "Place Order"

4.3. List all major activities

In this step, you will identify all key actions for the activity diagram. Since you have already specified these actions in the use case specification (both in the main flow of events and alternative flows), you can directly use them here. Each action from the flow of events should be represented as an **action node** in the diagram. Place the action nodes into the corresponding swimlanes/partitions and then connect them with arrows to indicate the sequence of events.

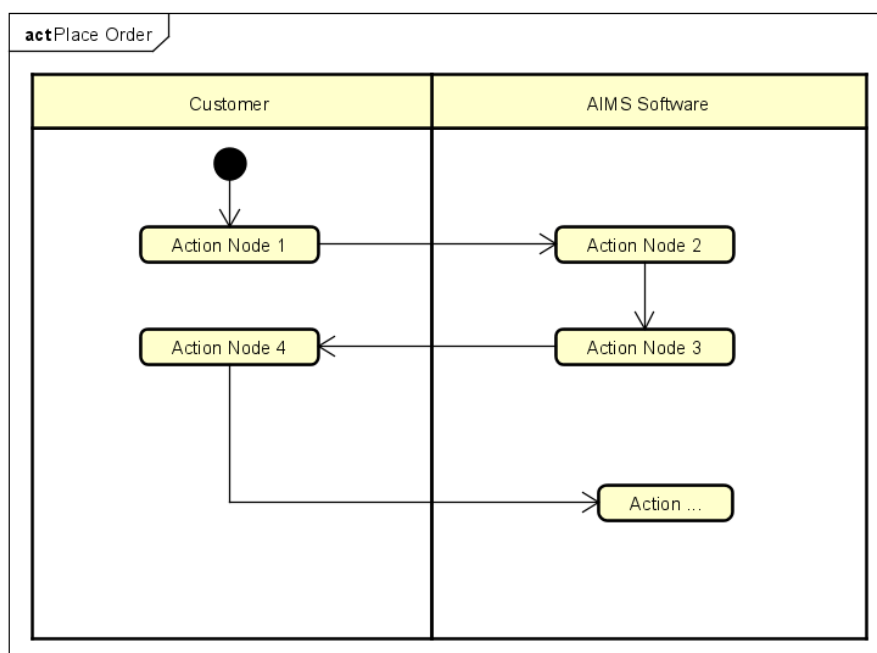


Figure 4. List of Action Nodes in Activity Diagram

4.4. Identify decision nodes

You have to determine where conditional branches occur. This happens when an event in the main flow does not proceed as expected and must divert to an alternative flow. You should answer the question: *What is the condition for each branch?* For the branch where the condition is true, the process continues along the main flow of events. For the branch where the condition is false, the flow diverts to the alternative events.

In use case “Place Order”, we can have some decision nodes. For example, when the software checks the stock quantity of the products in the shopping cart, if the condition is “sufficient quantity” (true branch), the customer will proceed to enter the delivery information. Otherwise, if the condition is “insufficient quantity” (false branch), the use case will either terminate or invoke another use case, such as “Update cart” (it depends on your concept).

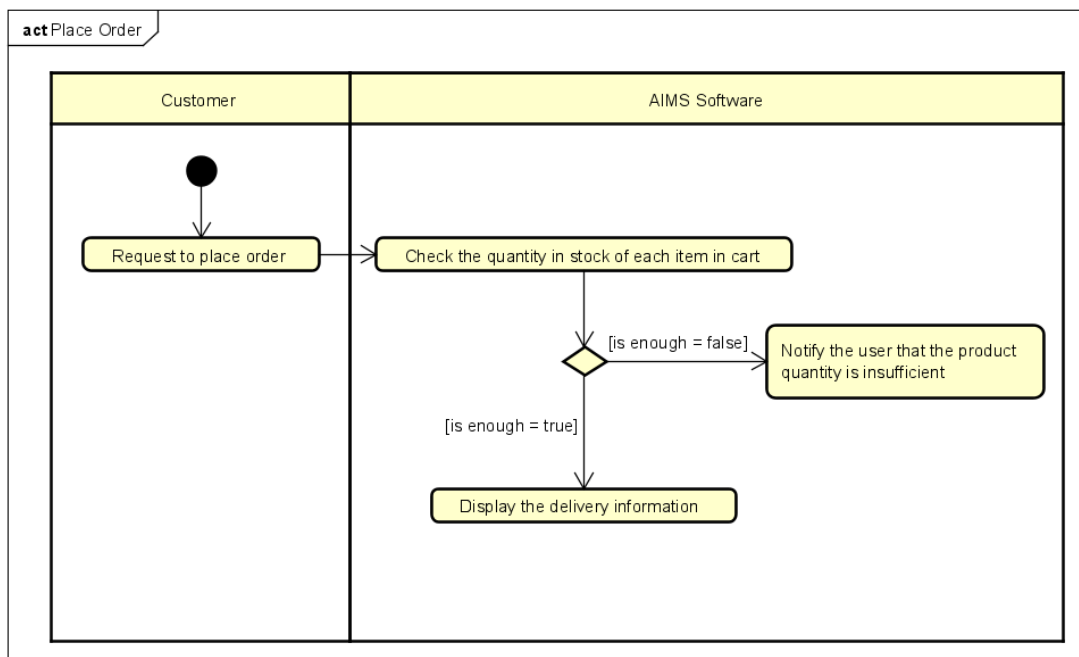


Figure 5. Decision Node in Activity Diagram

4.5. Identify parallel processes

Read all the actions in tip 3 carefully. You may recognize some actions that can occur simultaneously. In UML, **fork nodes** are used to split a single flow into multiple parallel flows, allowing activities to run concurrently. Conversely, **join nodes** are used to merge multiple parallel flows back into a single flow, ensuring that all parallel activities are completed before the process continues. When creating your

activity diagram, you can use fork and join nodes to clearly represent parallel processes. In use case “Place Order”, after a successful order has been placed, the AIMS software can *send email to customer* and *show order information* simultaneously.

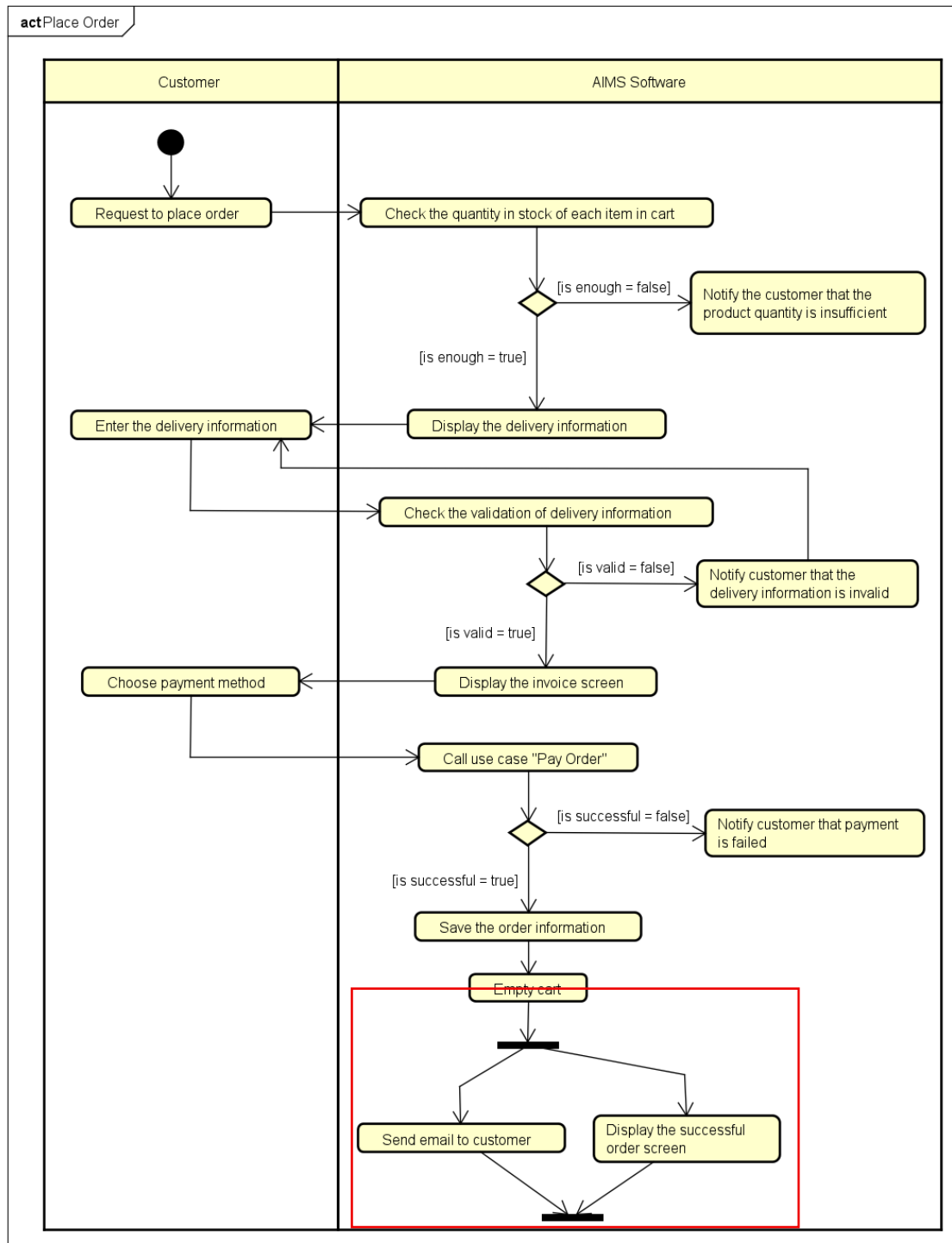


Figure 6. Use Join Nodes and Fork Nodes in Activity Diagram

4.6. Identify the end node(s)

Finally, you will determine the end node. Unlike the start node, an activity diagram **can have more than one end node** to represent different possible outcomes or termination points in the process.

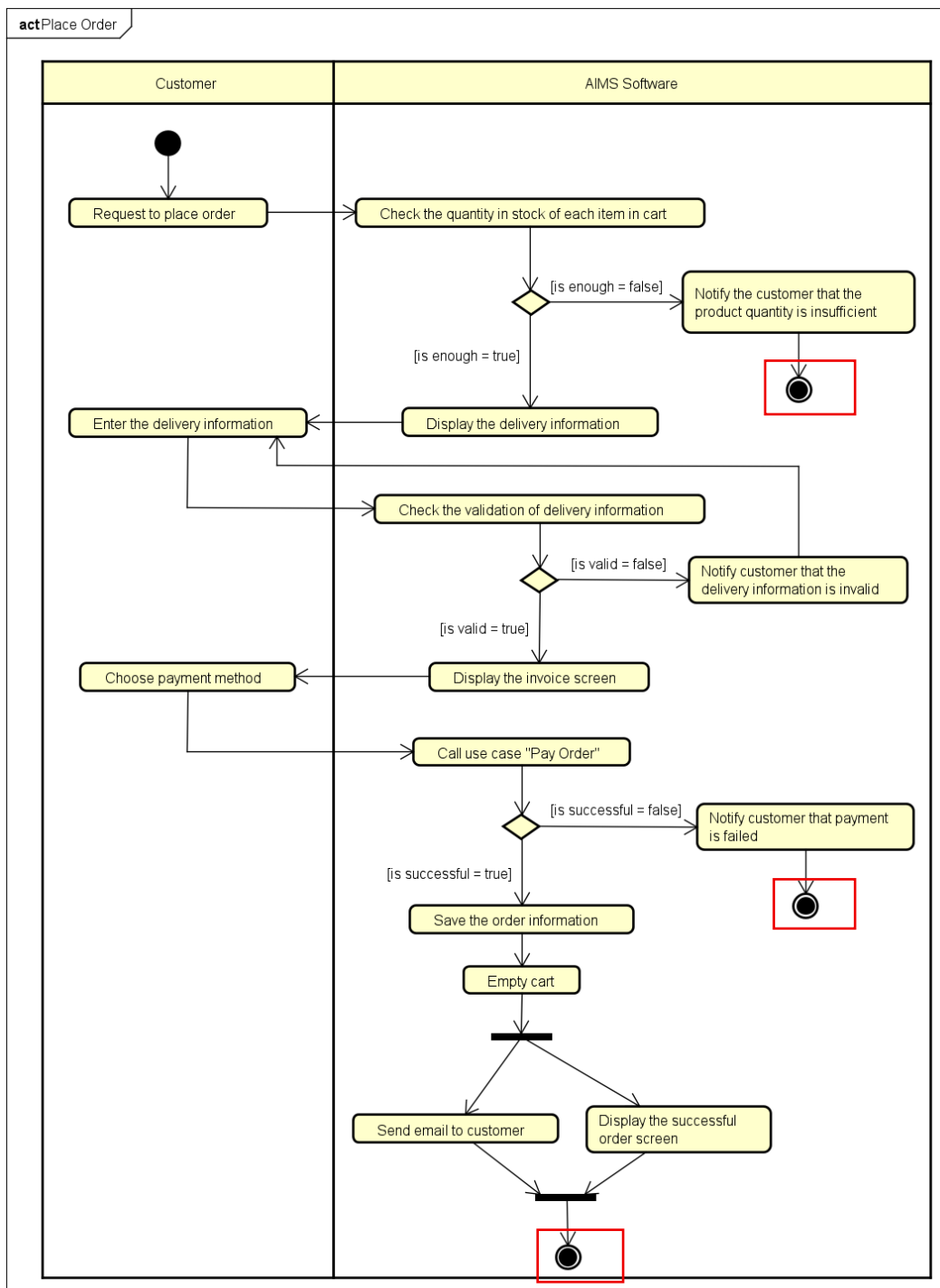


Figure 7. Activity Diagram for use case "Place Order"

All above images/diagrams in this guideline are merely examples, so *they may be lack or inaccurate with the problem statement*. You need to check carefully to ensure that your diagram(s) are consistent with your use case model.

Now, you will draw activity diagrams for the use cases assigned to you. You are required to create a directory named `RequirementAnalysis/ActivityDiagram/` to submit your work. You must submit both the Astah project file (.asta) and the exported image file (in PNG or JPG format). Don't forget to name the files according to the corresponding use case.

```
RequirementAnalysis/
└─ ActivityDiagram/
    └─ astah/
        └─ PlaceOrder.asta
        └─ ... (put your files here)
    └─ images/
        └─ PlaceOrder.png
        └─ ... (put your files here)
```

5. SOFTWARE REQUIREMENT SPECIFICATION

The final task in this lab is to complete the **Software Requirement Specification (SRS)** document. A template for this document has already been provided by the lecturer. Therefore, you should strictly follow the given template and its guidelines to complete all required sections of the SRS.

After completing all assigned tasks, you are required to create a directory named `RequirementAnalysis/SRS/` to submit your work. You must include both the Word file (DOC or DOCX format) and its corresponding exported PDF file.

To ensure easy identification between different teams and document versions, please follow the naming convention:

`[TEAM-NO]SoftwareRequirementSpecification-[VerXx]`

- `[TEAM-NO]`: your team number (e.g., TEAM-01)
- `[Ver-Xx]`: the current version of the document (e.g., Ver-1.0)

RequirementAnalysis/

└─ SRS/

└─ [TEAM-NO]SoftwareRequirementSpecification-[VerXx].docx

└─ [TEAM-NO]SoftwareRequirementSpecification-[VerXx].pdf

Table of contents	1
1 Introduction	2
1.1 Objective.....	2
1.2 Scope	2
1.3 Glossary	2
1.4 References	3
2 Overall Description	4
2.1 Survey.....	4
2.2 Overall requirements	4
2.3 Business process	4
3 Detailed Requirements	5
3.1 Use case 1	6
3.2 Use case 2	7
4 Supplementary specification	8
4.1 Functionality.....	8
4.2 Usability.....	8
4.3 Reliability	8
4.4 Performance.....	8
4.5 Supportability	8
4.6 Other requirements	8

Figure 8. Template for the SRS document